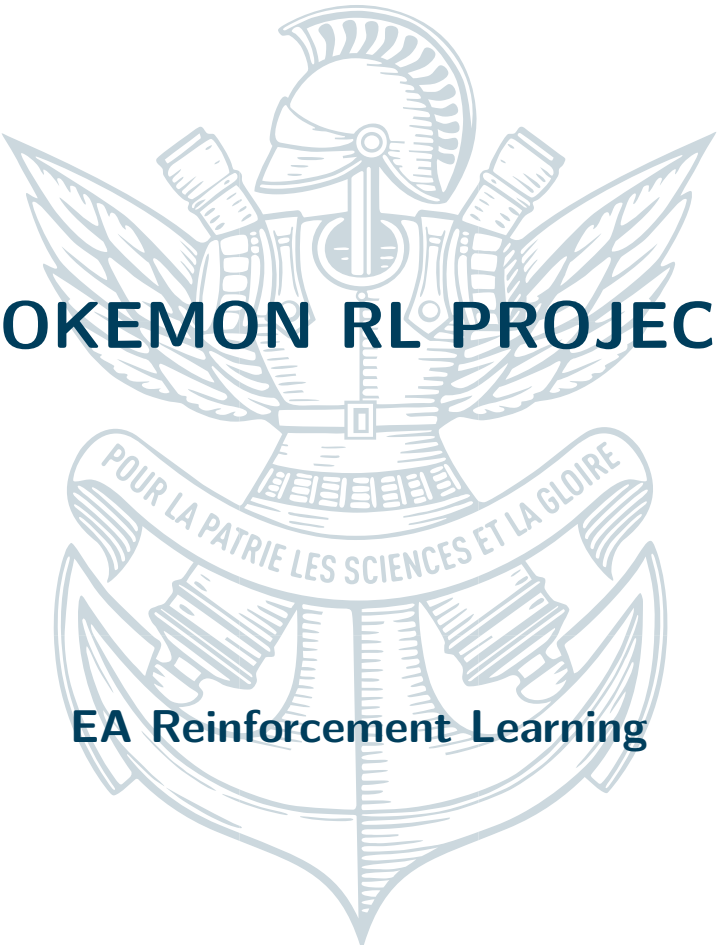




POKEMON RL PROJECT

EA Reinforcement Learning

7 mars 2026

Côme Rochas, Thomas Gastellu
Benjamin Selles, Leonel Gomes Abreu
Théo Clary



TABLE DES MATIÈRES

1	Introduction	3
2	Environment	4
3	Models	5
4	Training Setup	6
5	Experimental Results	7
6	Appendix	8

1

INTRODUCTION

In this project, we explore the application of reinforcement learning (RL) techniques to the domain of Pokemon battles. Our goal is to develop an RL agent capable of learning effective strategies for battling against a heuristic opponent.

The pokemon battle environment is a turn-based game where two players select actions (such as attacking, switching Pokemon, or using items) to defeat their opponent's team. The action space is also large, as there are multiple moves and strategies to consider at each turn. This environment presents three main challenges :

- **High-dimensional state and action spaces** : The large number of possible states and actions makes it difficult for the agent to learn effective policies.
- **Partial observability** : The agent does not have access to the full state of the environment, as it cannot see the opponent's Pokemon or their moves.
- **High stochasticity** : The outcome of actions can be highly variable due to factors such as move accuracy, critical hits, and random effects, making it challenging for the agent to learn consistent strategies.

Previous approaches range from simple scripted baselines to modern reinforcement learning and search-based agents. Common baselines include hand-crafted heuristics such as always picking the highest base-power move or switching when at a clear type disadvantage, as well as purely random policies used for benchmarking. Classical methods like tabular Q-learning and softmax exploration have been applied in simplified environments [1], showing consistent gains over random agents while remaining limited by state-space size. More recent work uses deep RL (DQN, A3C, PPO) on top of Pokémon Showdown via dedicated environments, often finding policy-gradient methods like PPO particularly effective in stochastic, high-dimensional settings. Beyond RL, some systems combine minimax-style lookahead with learned value functions or leverage large language models [2] and offline sequence models trained on battle logs, which excel at capturing long-horizon strategy and opponent modeling but trade off interpretability and computational cost.

We have implemented two PPO-based agents : a simple policy network and a more complex actor-critic architecture. The report is structured as follows : we first describe the environment 2., then we detail the models we implemented 3., followed by the training setup 4. and finally we present our experimental results 5.

2

ENVIRONMENT

The environment is implemented with `poke-env`, connected to a local Pokemon Showdown server. In our code, `PokeRLEnv` extends `SinglesEnv` (multi-agent), and is wrapped by `MaskableSingleAgentWrapper` to expose a standard single-agent `gymnasium.Env` interface for Stable-Baselines3.

At each decision step, `poke-env` provides a `Battle` object (state snapshot of the current turn). Our observation pipeline is :

$$\text{Battle state} \longrightarrow \text{embed_battle}(\text{battle}) \longrightarrow \mathbf{s}_t \in \mathbb{R}^{490}.$$

The `Battle` object is not a flat vector : it is a structured Python object with many fields. Our encoder reads only the fields needed for policy learning :

- active Pokemon on both sides (HP fraction, types, boosts, status),
- known moves of the active Pokemon,
- revealed team members on both sides (bench information),
- side conditions (entry hazards and screens),
- global effects (weather and field effects),
- availability flags for battle gimmicks (Dynamax, Mega, Z-Move, Tera).

This is still a partially observable setting : opponent information is only available when revealed during battle (e.g., observed Pokemon and seen moves).

The state vector is built by concatenating feature blocks in a fixed order :

$$\mathbf{s}_t = [\mathbf{s}^{\text{ally_active}}, \mathbf{s}^{\text{ally_moves}}, \mathbf{s}^{\text{opp_active}}, \mathbf{s}^{\text{opp_moves}}, \mathbf{s}^{\text{ally_bench}}, \mathbf{s}^{\text{opp_bench}}, \mathbf{s}^{\text{side}}, \mathbf{s}^{\text{weather}}, \mathbf{s}^{\text{field}}, \mathbf{s}^{\text{gimmick}}].$$

- **Ally active Pokemon (33)** : HP fraction (1), type multi-hot over 18 types, 7 normalized stat boosts (−6 to +6 scaled to $[-1, 1]$), and status one-hot (7).
- **Ally active moves (92)** : 4 fixed slots, each encoded with 23 values : move type one-hot (18), base power/250, accuracy/100, category bits (physical/special), and normalized PP $\text{current_pp}/\text{max_pp}$.
- **Opponent active Pokemon (33)** : same encoding as ally active Pokemon.
- **Known opponent moves (220)** : up to 10 known moves, 22 values each (same as above *without* PP, because opponent PP is not exposed by the server). Unknown slots are zero-padded.
- **Ally bench (40)** and **opponent bench (40)** : 5 slots per side, each slot has HP fraction (1) + status one-hot (7), for 8 values per slot. Missing/unrevealed slots are zero-padded.
- **Side conditions (14)** : for each side, 4 hazard values (Stealth Rock, Spikes layers, Toxic Spikes layers, Sticky Web) and 3 binary screen flags (Reflect, Light Screen, Aurora Veil).
- **Weather (8)** : one-hot over the supported weather states.
- **Field effects (6)** : one-hot over terrain/global field effects (Electric/Grassy/Misty/Psychic Terrain, Trick Room, Gravity).
- **Gimmick availability (4)** : $[\text{can_dynamax}, \text{can_mega_evolve}, \text{can_tera}, \text{can_z_move}]$.

Therefore, the final observation size is :

$$33 + 92 + 33 + 220 + 40 + 40 + 14 + 8 + 6 + 4 = 490.$$

All features are cast to `float32`. Continuous values are normalized (HP, boosts, power, accuracy, PP), categorical values are encoded as one-hot/multi-hot vectors, and variable-length information (moves, bench, revealed opponent data) is converted to fixed-size blocks by zero-padding.

3

MODELS

Our core model is **masked PPO**, implemented with **MaskablePPO** from **sb3-contrib** (Stable-Baselines3 ecosystem), with **RecurrentPPO** available as an optional variant. Using an established SB3 implementation was an intentional choice : it gives us a reliable optimization pipeline, well-tested training code, and straightforward integration with our Gymnasium-compatible environment.

This choice is also motivated by the structure of Pokemon battles : the state is high-dimensional, transitions are stochastic, and the action space is discrete with many context-dependent invalid actions. In this regime, an on-policy actor-critic approach with conservative policy updates is generally more stable than value-based baselines, and action masking improves sample efficiency by restricting exploration to legal decisions only.

Operationally, at each turn the policy receives the encoded state vector $\mathbf{s}_t$ from Section 2, then outputs a categorical distribution over actions. The environment mask is applied so only legal actions remain selectable ; the policy samples (or chooses) an action from that masked distribution. In parallel, the critic estimates $V_\phi(\mathbf{s}_t)$, used to compute an advantage estimate $\hat{A}_t$ for policy optimization. This actor-critic decomposition is useful here because it separates action selection from state evaluation : the actor improves tactical choice quality, while the critic provides a lower-variance learning signal.

PPO updates the policy using the probability ratio $r_t(\theta) = \pi_\theta(a_t | \mathbf{s}_t) / \pi_{\theta_{\text{old}}}(a_t | \mathbf{s}_t)$, with clipped objective :

$$L^{\text{clip}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right].$$

The clipping term (cut-off) prevents overly large policy updates in one step. The final training loss also includes a value-function term and an entropy bonus to keep exploration.

In practice, the clipped ratio term is critical : without it, policy updates can become too aggressive, especially in noisy environments where short-term returns fluctuate strongly from one rollout to another. The clipped objective keeps updates within a trust region around the previous policy, which tends to produce more stable training dynamics.

For completeness, we also implemented additional approaches in the same training framework (notably DQN, and the recurrent PPO variant). However, these alternatives were not evaluated as extensively as the default masked PPO setup, so our main analysis and conclusions focus on the SB3 masked PPO model.

In short, the policy learns to assign probability mass to strong legal actions while remaining conservative between updates, which is particularly important in noisy, partially observable battles. Implementation details (network architecture and hyperparameters) are reported in Appendix 6.

4 TRAINING SETUP

5 EXPERIMENTAL RESULTS

6 APPENDIX

MODEL ARCHITECTURE

The PPO variants are configured with an MLP policy : `MlpPolicy` with `net_arch = [256, 256]`. The policy and value heads both operate on the 490-dimensional observation vector. The default algorithm is `MaskablePPO`; `RecurrentPPO` can be selected as an alternative through Hydra (`algorithm=recurrentppo`).

PPO HYPERPARAMETERS USED

The current parameters from `pokerl/configs/models/ppo.yaml` are :

- learning rate : 1×10^{-4}
- rollout length `n_steps` : 1024
- mini-batch size : 128
- optimization epochs per rollout : 5
- discount factor γ : 0.99
- GAE parameter λ : 0.95
- clipping range ϵ : 0.2
- entropy coefficient : 0.02
- value-function coefficient : 0.5
- max gradient norm : 0.5

TRAINING DEFAULTS

Default training configuration (`pokerl/configs/training/default.yaml`) : 10^6 timesteps, heuristic op-ponent for training, evaluation every 10,000 steps, and 50 evaluation episodes per checkpoint.

RÉFÉRENCES

- [1] Kalose, Akshay and Kaya, Kris and Kim, Alvin. 2018. Optimal battle strategy in pokemon using reinforcement learning. *Web* : <https://web.stanford.edu/class/aa228/reports/2018/final151.pdf>.
- [2] Jain, Daksh and Jain, Aarya and Desai, Ashutosh and Verma, Avyakt and Bhanuka, Ishan and Narang, Pratik and Kumar, Dhruv. 2025. Large Language Models as Pok\`emon Battle Agents : Strategic Play and Content Generation. *arXiv preprint arXiv :2512.17308*.